

DevOpsAI - Advanced Industry-Ready Project

Complete conversation notes: project recommendation, architecture, features, technology stack, execution environment, development workflow, and roadmap

This PDF consolidates the full substance of the project discussion without intentionally omitting any project point. It includes the recommendation itself and the follow-up explanation of where and how to execute it.

1. User Goal

The goal is to upskill using already-learned knowledge while building an advanced project that matches present industry requirements. The recommendation was to avoid another basic CRUD app, e-commerce clone, simple chatbot, or AI resume analyzer as the main portfolio project, because those would not stretch the current skill set enough.

The suggested direction is one serious, production-style project that combines Java, Spring Boot, REST APIs, SQL/DBMS, React, JavaScript, DSA, Git/GitHub, and AI/GenAI exposure, while adding production engineering, cloud, DevOps, observability, and system design.

2. Number-One Project Recommendation

Project: DevOpsAI - AI-Powered Developer Productivity & Code Intelligence Platform

Concept: a mini combination of GitHub + Jira + AI coding assistant + automated code review + developer analytics.

The platform would allow a development team to create projects, create tasks, connect GitHub, analyze commits and pull requests, use AI for code review, automatically generate tests, track bugs, monitor project health, and ask an AI agent questions about the codebase.

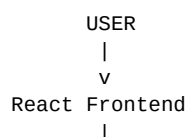
Create a project -> create tasks -> connect GitHub -> analyze commits/PRs
-> AI reviews code -> automatically generate tests -> track bugs
-> monitor project health -> ask an AI agent about the codebase

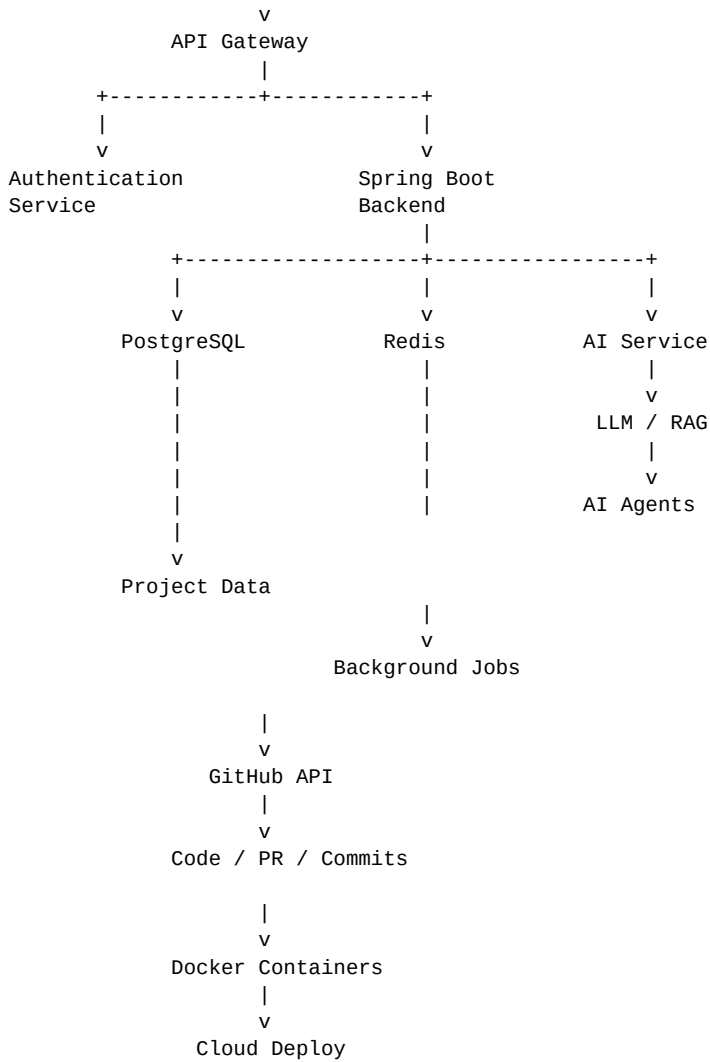
3. Example Dashboard

DevOpsAI	User
Project: SmartHealth	
[23 Tasks] [7 PRs] [3 Bugs]	
Code Quality	87%
Test Coverage	78%
Security Risk	LOW
AI Code Issues	4
Recent Activity	
- PR #42 analyzed	
- Authentication bug detected	
- 6 tests generated	
- Deployment successful	

The purpose of this dashboard is to make the project feel like a real developer productivity SaaS platform rather than a college demo.

4. High-Level Architecture





The main learning value is system design and integration, not just writing isolated code.

5. Feature 1 - Authentication and RBAC

Build proper authentication and authorization.

```

ADMIN
|
PROJECT_MANAGER
|
DEVELOPER
|
VIEWER
  
```

- JWT authentication
- Refresh tokens
- Password hashing
- Role-based authorization
- Project-level permissions

- Secure API endpoints

```

POST /api/auth/register
POST /api/auth/login
POST /api/auth/refresh
GET /api/users/me
GET /api/projects
POST /api/projects
PUT /api/projects/{id}
  
```

This directly uses Spring Boot, REST API, and security knowledge.

6. Feature 2 - Project Management

Build a lightweight Jira-like project management module.

```
Project
|
+-- Epic
+-- Task
+-- Bug
+-- Sprint
```

```
TODO -> IN_PROGRESS -> CODE_REVIEW -> TESTING -> DONE
```

Use DSA inside the product, not only in coding practice. A priority queue can help order critical bugs, high-priority tasks, normal tasks, and low-priority tasks.

```
priority score = severity + business impact + deadline + dependency
```

This gives a practical story for how data structures can support real product behavior.

7. Feature 3 - GitHub Integration

```
Repository
|
v
Commits
|
v
Pull Requests
|
v
Changed Files
|
v
Code Analysis
```

Example workflow: a developer creates PR #42, DevOpsAI detects the pull request, fetches changed files, sends them for AI analysis, generates a review, stores the results, and displays the findings on the dashboard.

8. Feature 4 - AI Code Reviewer

This should be one of the flagship features.

```
public User getUser(Long id) {
    return repository.findById(id).get();
}
```

AI finding:

Potential issue - `Optional.get()` can throw `NoSuchElementException`.

Recommendation - use `orElseThrow()`.

Review categories should include:

- Bug
- Security
- Performance
- Maintainability
- Code Style
- Best Practice

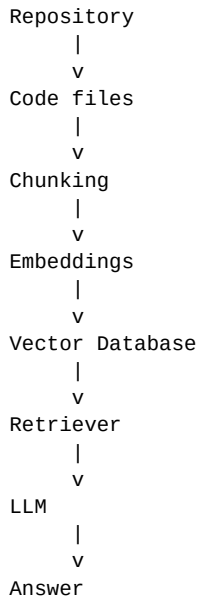
This is more impressive than simply calling an LLM API for a generic chatbot.

9. Feature 5 - RAG-Based Codebase Assistant

Build a retrieval-augmented assistant that answers questions about the connected repository.

Example question: Where is authentication handled?

The assistant should locate files such as AuthController.java, AuthService.java, JwtService.java, and SecurityConfig.java, and explain the login flow.



MongoDB Vector Search is a natural fit because it has already been explored. pgvector can be evaluated later.

10. Feature 6 - AI Agents

Do not stop at RAG. Add specialized agents with controlled tool access.

Agent 1 - Code Review Agent

Analyze PR -> Find bugs -> Find vulnerabilities -> Suggest improvements

Agent 2 - Test Agent

Read changed code -> Identify scenarios -> Generate test cases -> Generate JUnit tests

Agent 3 - Documentation Agent

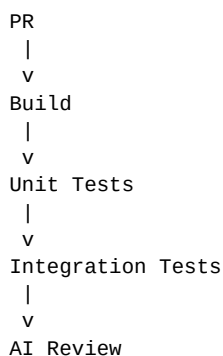
Read code -> Understand APIs -> Generate documentation

Agent 4 - Debugging Agent

Stack trace -> Relevant source files -> Recent commits -> Related logs
-> Possible root causes -> Suggested fix

The agents should not execute everything blindly; they should operate with permissions, validation, and human oversight.

11. Feature 7 - Automated Testing



```
|
v
Security Scan
|
v
Deploy
```

- JUnit
- Mockito
- Spring Boot Test
- Postman / Newman

Later, add GitHub Actions so tests and builds run automatically. This makes testing part of the delivery pipeline, not an afterthought.

12. Feature 8 - Security Scanner

Add security-oriented checks to the platform.

- Dependency vulnerabilities
- SQL injection risks
- Hardcoded secrets
- Weak authentication
- Insecure APIs

Security Report

CRITICAL	1
HIGH	2
MEDIUM	5
LOW	7

13. Feature 9 - Developer Analytics

Developer Productivity

Commits	43
Pull Requests	12
Average PR Time	4.2h
Bug Resolution	7.3h
Test Coverage	81%
Code Quality	89%

Useful analytics and charts: commit frequency, bug trends, PR cycle time, code quality, and sprint velocity. These make the application resemble a real SaaS product.

14. Feature 10 - Cloud Deployment

Do not leave the final application only on localhost. Deploy it after the local version is stable.

```
Frontend
React
|
v
Cloud hosting
```

```
Backend
Spring Boot
|
v
Docker
|
v
Cloud
```

Database: PostgreSQL
Cache: Redis
AI: LLM API
Vector DB: MongoDB Atlas / pgvector
CI/CD: GitHub Actions

Kubernetes can be added later. Docker + CI/CD + cloud deployment should be part of the final version.

15. Complete Technology Stack

Layer	Technology
Frontend	React
UI	Tailwind CSS
Backend	Java + Spring Boot
API	REST
Authentication	JWT + Spring Security
Database	PostgreSQL
ORM	Spring Data JPA
Cache	Redis
AI	LLM API
RAG	Embeddings + Vector DB
Vector DB	MongoDB Vector Search / pgvector
Testing	JUnit + Mockito
API Testing	Postman
Version Control	Git + GitHub
CI/CD	GitHub Actions
Containerization	Docker
Cloud	AWS / Azure
Monitoring	Spring Boot Actuator + Prometheus/Grafana
Documentation	Swagger/OpenAPI

16. Development Versions and Phases

Version 1 - Full Stack Foundation

React + Spring Boot + PostgreSQL + JWT

Build login, projects, tasks, bugs, sprints, and dashboard.

Version 2 - Production Backend

Add Spring Security, Redis, pagination, sorting, filtering, global exception handling, validation, Swagger, logging, JUnit, and Mockito.

Version 3 - GitHub Integration

Add GitHub OAuth, GitHub API, webhooks, repository analysis, commit tracking, and pull request tracking.

Version 4 - AI

Add AI code review, AI test generation, AI documentation, and a RAG codebase assistant.

Version 5 - Agentic AI

Add code review, testing, debugging, and documentation agents.

Version 6 - DevOps

Add Docker, CI/CD, cloud, monitoring, logs, metrics, and alerts.

Version 7 - Monitoring and Optimization

Improve performance, observability, scalability, cost control, reliability, and production readiness.

17. What You Should Be Able to Say in an Interview

I built an AI-powered developer productivity platform using React and Spring Boot that integrates with GitHub to analyze repositories and pull requests. I implemented JWT-based authentication, role-based access control, PostgreSQL persistence, Redis caching, RAG-based codebase search, AI-assisted code reviews and test generation, automated testing, Docker-based deployment and CI/CD.

This statement naturally opens interview discussion about REST API design, JWT, Spring Security, JPA, SQL optimization, indexing, caching, Redis, transactions, concurrency, DSA, Git, GitHub APIs, webhooks, RAG, embeddings, vector databases, LLMs, agents, prompt engineering, hallucinations, evaluation, testing, Docker, CI/CD, cloud, monitoring, and system design.

18. Why This Project Fits the Existing Knowledge

Already-known or explored areas include Java, Spring Boot, REST APIs, SQL/DBMS, React, JavaScript, Git/GitHub, DSA, GenAI, and vector search. The missing skill is not another programming language; it is integration.

Move from: I know Spring Boot. To: I can architect and ship a production-style application using Spring Boot.

Move from: I know GenAI. To: I can integrate AI into a real software workflow with retrieval, tools, evaluation, and guardrails.

The strategy is to become a software engineer who can build AI-powered systems, not someone who only knows how to call an LLM API.

19. What Not to Do

Do not try to learn Kubernetes, Kafka, microservices, Terraform, AWS, LangChain, LangGraph, Redis, MongoDB, Docker, React, and Spring Boot all at once. That creates the classic trap of opening many tutorials and completing no project.

```
PHASE 1
Spring Boot + React + PostgreSQL
      |
      v
PHASE 2
Security + Testing + Redis
      |
      v
PHASE 3
GitHub integration
      |
      v
PHASE 4
RAG + AI
      |
      v
PHASE 5
Agents
      |
      v
PHASE 6
Docker + CI/CD + Cloud
      |
      v
PHASE 7
```

20. Portfolio Comparison

Project	Industry Value	Difficulty	Portfolio Value
Basic E-commerce	2/5	2/5	2/5
Chatbot	2/5	2/5	2/5
CRUD Management System	2/5	1/5	2/5
RAG Document Chatbot	4/5	3/5	4/5
AI Resume Analyzer	3/5	2/5	3/5
DevOpsAI Platform	5/5	5/5	5/5
Distributed AI Platform	5/5	5/5	5/5

Recommendation: DevOpsAI is the best fit because it sits in the sweet spot between existing knowledge and the next skills needed for industry readiness.

21. Where to Execute the Project

Execute it locally on a Windows laptop first using VS Code + WSL2 + Docker Desktop, then deploy to the cloud after the core system works. A powerful cloud server is not necessary on day one.

```
YOUR WINDOWS LAPTOP
|
+-- VS Code
+-- WSL2 / Ubuntu
+-- Docker Desktop
+-- Git + GitHub
    |
    v
  DevOpsAI Project
    |
    +-- React Frontend
    +-- Spring Boot Backend
    +-- PostgreSQL
    +-- Redis
    +-- MongoDB / Vector DB
    +-- AI API
```

22. Frontend Execution

Use VS Code for React + Vite.

```
devopsai/
+-- frontend/
   +-- src/
   +-- components/
   +-- pages/
   +-- services/
```

Run the frontend with:

```
npm run dev
```

A typical local URL is <http://localhost:5173>.

23. Backend Execution

Use IntelliJ IDEA or VS Code. IntelliJ IDEA was recommended for the Spring Boot backend because it provides strong support for Maven, debugging, JPA, database inspection, refactoring, dependency management, Git, and REST development.

VS Code can still be used for everything if preferred; there is no need to install multiple IDEs just for the sake of it.

```
devopsai/
+-- backend/
   +-- controller/
   +-- service/
   +-- repository/
   +-- entity/
   +-- dto/
   +-- security/
   +-- exception/
   +-- config/
```

24. PostgreSQL and Redis Execution

Run PostgreSQL and Redis through Docker rather than manually installing every infrastructure service on Windows. MongoDB can also be containerized for local development, while MongoDB Atlas is a good managed option.

```
docker compose up -d
```

This gives practical containerization experience and keeps the environment reproducible.

25. AI Execution

Do not run a large local language model at the beginning. Use an API-based LLM first.

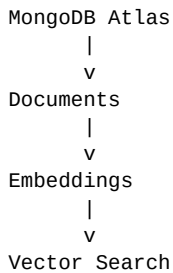
Spring Boot -> AI Service -> LLM API

If a suitable NVIDIA GPU is available later, local GPU-backed containers through WSL2/Docker can be explored, but this is not required for the first implementation.

26. Vector Database Options

Option A - MongoDB Vector Search

This is the smoother initial choice because MongoDB Vector Search has already been explored.



Option B - PostgreSQL + pgvector

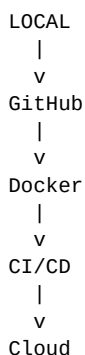
This is also an excellent choice because it can keep users, projects, tasks, bugs, and embeddings within the PostgreSQL ecosystem.

Suggested first version: PostgreSQL for application data and MongoDB Atlas for code/document vector search. Later, compare or migrate to pgvector if useful.

27. Testing Setup

- Backend: JUnit, Mockito, Spring Boot Test
- API: Postman
- Frontend: Vitest / React Testing Library
- Later: GitHub Actions for automatic tests, builds, Docker images, and deployment

28. Cloud Comes Last



Do not pay for expensive infrastructure while still debugging basic local issues. Cloud deployment should follow a stable local implementation.

Possible final arrangement: React on cloud hosting, Spring Boot in a cloud server/container, PostgreSQL as a managed database, Redis as a managed cache, an LLM API for AI, and MongoDB Atlas for vector search.

29. Recommended Project Folder

Keep the project inside the WSL2/Ubuntu Linux filesystem for a smooth WSL + Docker development workflow.

~/projects/devopsai/

```

devopsai/
|
+-- frontend/
+-- backend/
+-- ai-service/
+-- infrastructure/
|   +-- docker/
|   +-- docker-compose.yml
+-- docs/
+-- scripts/
+-- README.md

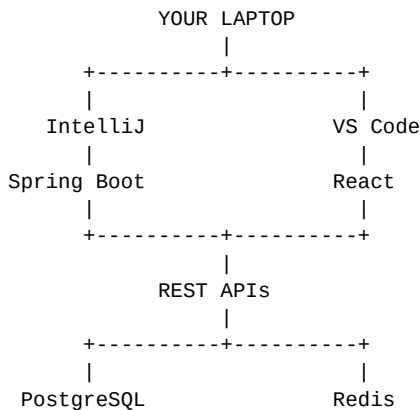
```

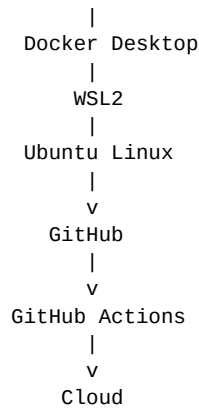
This is better than scattering project files across random Desktop folders with multiple final versions.

30. Recommended Development Environment

Requirement	Choice
Host OS	Windows
Linux environment	WSL2 + Ubuntu
Java IDE	IntelliJ IDEA
Frontend IDE	VS Code
Backend	Spring Boot
Java	Modern supported LTS
Build	Maven
Frontend	React + Vite
Database	PostgreSQL
Cache	Redis
Vector DB	MongoDB Atlas
AI	LLM API
API testing	Postman
Version control	Git + GitHub
Containers	Docker Desktop
CI/CD	GitHub Actions
Deployment	Cloud
Monitoring	Spring Boot Actuator + Prometheus/Grafana

31. End-to-End Development Workflow





32. Clean Setup Order Before Coding

- Step 1 -> Check Java
- Step 2 -> Check Maven
- Step 3 -> Check Node/npm
- Step 4 -> Check WSL2
- Step 5 -> Install/configure Docker Desktop
- Step 6 -> Create GitHub repository
- Step 7 -> Create the project skeleton
- Step 8 -> Build the backend first
- Step 9 -> Add React
- Step 10 -> Add AI/RAG

The key recommendation is to establish a clean development environment first, then build the application incrementally. This avoids wasting time on environment problems while also trying to debug application code.

33. Final Direction

DevOpsAI is intended to be a career-building project rather than a GitHub decoration. Build it in layers: strong backend foundation first, then security and testing, then GitHub integration, then AI/RAG, then agents, then Docker/CI/CD/cloud, and finally observability and optimization.

The central idea is integration. Use existing Java/Spring/React knowledge as the foundation and add production engineering, AI, DevOps, cloud, and system design in a controlled sequence.

34. Original Follow-Up Question Covered

Where should I execute this project?

Answer: locally first on a Windows laptop using WSL2/Ubuntu, Docker Desktop, IntelliJ IDEA for Spring Boot, VS Code for React and AI experiments, PostgreSQL/Redis through Docker, MongoDB Atlas or pgvector for vector search, and Git/GitHub for version control. Deploy to the cloud only after the local system becomes stable.